CSA17 – ARTIFICIAL INTELLIGENCE

ASSIGNMENT 2 – CO2

CO2 Coverage: Informed search strategies – Greedy best-first search – A* search – Heuristic functions – Local search algorithms and optimisation problems – Online search agent – Constraint satisfaction problems – Game playing: adversarial search, minimax algorithm, alpha-beta pruning, evaluation functions.

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: _____

TASK 1 – Greedy Best-First and A* Under Different Map Conditions (6 Marks)
Problem Context – Autonomous Campus Shuttle

A campus shuttle plans routes on grid maps of the campus. Obstacles (construction zones, closed roads) are placed in different patterns.

The development team is considering:

Uniform-Cost Search | Greedy Best-First Search | A* Search

However, the maps may be open, cluttered, maze-like or contain dead-end traps.
Your Challenge

Investigate whether one of the three algorithms should always be preferred or whether the appropriate choice depends on the characteristics of the map.

Before implementation:

• Predict how the three algorithms will behave under different map conditions • Explain the algorithmic reason for your prediction

• Design your own admissible heuristic and justify its admissibility using a relaxed version of the problem

Design two deliberately contrasting maps and one additional edge-case map of your own that will expose meaningful behavioural differences among the algorithms.

Implement the three algorithms and record appropriate measures such as:

• Nodes expanded and nodes generated

• Maximum frontier size

• Path cost and whether the path is optimal

• Execution time, where useful

Compare your prediction with the observed behaviour, with code and outputs as proof.

Design Decision

The operations team reports that approximately 95% of shuttle trips are on open roads with only occasional detours around obstacles. Determine which algorithm you would recommend and defend your decision.

Evidence Expected: Reasoned prediction, a justified heuristic, three deliberately designed maps, implementation of the three algorithms, metric tables, comparison of predicted and observed behaviour and an evidence-based recommendation.

TASK 2 – When Is an Online Search Agent an Appropriate Engineering Choice? (4 Marks) Problem Context – Exploration Rover in an Unmapped Facility

A rover must find a charging dock inside a facility whose map is not available. It can sense only the cells next to it, and it learns the map as it moves. The rover uses an online depth-first search (and, optionally, LRTA*).

Your Challenge

Design experiments to investigate the online agent under:

• Targets at different distances from the start

• Unreachable targets

• Different maze sizes and dead-end densities

Record the number of moves, revisited cells and the competitive ratio (cost travelled ÷ optimal cost with full map knowledge).

Then answer:

Under what operating conditions is an online search agent a reasonable engineering choice, and when would you stop recommending it?

Consider both the size of the environment and the number of repeated missions in the same facility (for example, learning across trials with LRTA*).

Changed Condition

The organisation now deploys 20 rovers across 20 facilities, and the maps remain unchanged for months. Re-evaluate your original recommendation.

Evidence Expected: Designed experiments, move-count and competitive-ratio evidence, analysis of successful and unsuccessful searches, an evidence-based operating boundary and a justified response to the changed condition.

TASK 3 – Engineer the Behaviour of Alpha-Beta Pruning (6 Marks)

Problem Context – Game Engine for a Board Game

A game company uses minimax with alpha-beta pruning for its two-player board game engine, and needs to understand how much work the pruning really saves.

Your Challenge

Instead of merely implementing the algorithm, investigate how the structure of the game tree influences computational effort.

Design:

1. One game tree in which alpha-beta pruning performs relatively little work (many cutoffs) 2. One game tree deliberately engineered so that almost no cutoffs occur

3. One additional test condition of your own choice (for example, random move order or a tree with equal leaf values)

Before executing each test:

• Predict the expected number of nodes visited

• Explain the algorithmic reason for your prediction

Implement minimax and alpha-beta and record the number of nodes visited and the number of cutoffs. **Inquiry**

Compare prediction with experimental result. Explain:

• Why one tree requires substantially more work than another

• Which characteristics of move ordering create or prevent cutoffs

• What this reveals about evaluating an algorithm using only the branching factor and depth

Evidence Expected: Correct implementation, three designed trees, predictions before execution, node and cutoff counts, and a reasoned explanation of the differences.

TASK 4 – Minimax and the Cost of Examining Every Possibility (6 Marks)
Problem Context – Turn-Based Strategy Game

A game AI must choose its next move by looking ahead in a game with an average of *b* legal moves per turn.

Your Challenge

Design a depth-limited minimax solution with an evaluation function of your own. Before implementation, determine:

• How many positions are examined for lookahead depth *d*

• How the evaluation function is used when the search is cut off

• How the number of terminal evaluations changes with *d*

Derive the number of nodes mathematically. Implement your solution and test it using at least three meaningfully different board sizes or depths.

Record:

• Number of nodes expanded and evaluation-function calls

• Execution time, where useful

• Quality of the move chosen (for example, win rate over 20 games against a random player) Compare the theoretical growth with experimental behaviour.

Scalability Inquiry

Define an acceptable response time for your experimental environment. Using your evidence, determine approximately when plain minimax becomes impractical. Identify where the inefficiency originates and describe what a redesigned strategy would need to avoid or reduce. You are not required to implement the redesign in this task.

Evidence Expected: Algorithm design, mathematical node-count derivation, implementation, experimental growth evidence, a defined acceptability criterion and a reasoned identification of the limitation.

TASK 5 – Derive a Constraint-Propagation Algorithm from a Principle (7 Marks)
Problem Context – University Exam Timetabling

Exams that share students cannot be placed in the same slot. Each exam (variable) has a set of possible slots (domain).

You are required to develop a constraint-propagation solution.

Design Starting Point

You are given only the following principle:

A value may be removed from a variable's domain if some neighbouring variable has no value that is compatible with it.

You are not provided with the complete algorithm.

Your Challenge

Convert this principle into a computational procedure. Determine:

• How the constraints are represented

• How a value without support is detected

• Which other pairs must be re-examined after a value is removed

• What condition indicates that the problem has no solution

• How the propagation is combined with backtracking search

Develop your own pseudocode before implementation.

Test Your Design

Test the algorithm using:

• A timetable instance in which propagation solves or detects failure without any search • A second instance designed by you to challenge your algorithm

Visualise the shrinking domains. Compare backtracking with and without your propagation on nodes visited, backtracks, constraint checks and domain wipe-outs, and analyse how the work grows as the number of exams increases.

Correctness Inquiry

Explain:

Why does removing an unsupported value never remove a value that belongs to a solution?

Your explanation should be based on reasoning rather than program output.

Evidence Expected: A student-derived propagation rule, pseudocode, implementation, designed test cases, computational analysis, domain-reduction visualisation and a reasoned correctness argument.

TASK 6 – Exhaustive Search, Local Search and Combinatorial Growth (8 Marks) Problem Context – Delivery Tour Planning

A courier must visit *n* customers exactly once and return to the depot, minimising the total travel distance. **Your Challenge**

Design an exhaustive-search strategy. Determine how candidate tours will be represented, generated, evaluated and how the best tour will be retained. Do not begin from supplied pseudocode.

Correctness Check: Create a small instance whose tours can be independently verified and use it to establish that your algorithm finds the true optimum.

Growth Investigation

Before experimentation, derive/predict how the number of candidate tours grows with *n*. Use at least three problem sizes to validate your prediction experimentally.

Local Search Alternative

Design and implement a local-search method (for example, hill climbing with 2-opt moves with random restarts, or simulated annealing). For sizes where the exhaustive optimum is known, record the percentage by which your local-search tour exceeds the optimum, and the time taken, over at least 20 runs.

Inquiry

Explain why adding a small number of customers produces a dramatic increase in computational effort, relating your explanation to the solution space. Explain why local search can fail to reach the optimum (local minima, plateaus).

Deployment Decision

The planner must produce a plan within a fixed response time selected and justified by you. Estimate the largest *n* for which you would deploy exhaustive search, and the point at which you would switch to local search.

Evidence Expected: Student-designed exhaustive enumeration, correctness verification, growth prediction and validation, local-search quality and time evidence, and an evidence-based deployment boundary.

DAY-4 DESIGN CHALLENGE

TASK 7 – Diagnose, Redesign and Validate (13 Marks)

Review the evidence you generated while working on Tasks 1–6. Select **ONE** solution that demonstrates a significant computational limitation as its problem size or operating requirement increases.

You are not permitted simply to replace the solution with a more advanced textbook algorithm and state that its complexity is better. Understand the computational problem first and then develop an appropriate redesign.

A. Diagnose the Computational Bottleneck – 2 Marks

Using your own evidence, identify where most computational effort is spent, which operation grows significantly, whether work is unnecessarily repeated and which characteristic of the original algorithm causes the limitation.

B. Define the Design Objective – 2 Marks

Before proposing a new solution, state what your redesign must achieve (for example, reduce nodes expanded, exploit move ordering, reuse previously computed information, reduce constraint checks). Specify a measurable objective.

C. Design the Revised Computational Strategy – 4 Marks

Explain what you have changed, what property your new design exploits, which operations are avoided, whether preprocessing or extra memory is introduced and what new trade-offs are created. Provide pseudocode or a design representation. Marks are based on the quality of the reasoning, not on naming a known algorithm.

D. Predict Before Implementing – 1 Mark

Predict how the redesigned solution should differ from the original and identify the performance measure in which you expect improvement.

E. Implement and Validate the Redesign – 2 Marks

Implement a prototype and compare the original and redesigned solutions on the same or equivalent test conditions using nodes expanded, constraint checks, evaluation calls, path quality or other suitable measures, with code and output as proof.

F. Defend the Final Design – 2 Marks

Evaluate whether the redesign achieved its objective: whether the prediction was supported, the magnitude of improvement, new costs, and situations in which the original may still be preferable. A redesign that does not outperform the original is not automatically unsuccessful if you correctly analyse why.

MARK DISTRIBUTION – ASSIGNMENT 2

Task	**Assessed Area**	**Marks**
Task 1	Uniform-cost, Greedy and A* search	6
Task 2	Online search agent	4
Task 3	Alpha-beta pruning	6
Task 4	Minimax and evaluation functions	6
Task 5	Constraint propagation and CSP	7
Task 6	Exhaustive and local search, Optimization	8
Task 7	Day-4 Design Challenge – Diagnose, Redesign & Validate	13
	TOTAL	**50**

Submission Requirements

Submit:

• Your assignment report containing the required evidence for Tasks 1–7

• Source code for all implemented solutions

• Raw experimental data used to produce your results

• Tables/graphs used in your analysis, where applicable

Operation counts (nodes expanded, constraint checks, evaluation calls) should be treated as primary experimental evidence. Execution time may be reported as additional evidence but should not be the sole basis of your conclusions.

Assessment Verification

Students may be asked to explain any part of their algorithmic design, source code, test-data design, experimental results, predictions, analysis or design challenge during a brief viva. The student should be able to explain and defend the evidence submitted as their own work.

Overall Assessment Expectation

This assignment does not primarily assess your ability to reproduce textbook algorithms. It assesses your ability to:

Predict → Design → Implement → Measure → Analyse → Evaluate → Diagnose → Redesign → Validate

Your conclusions and design decisions must be supported by your own mathematical reasoning and experimental evidence.

TASK 1 – Greedy Best-First and A* Under Different Map Conditions (6 Marks)

1. Aim

To compare Uniform-Cost Search (UCS), Greedy Best-First Search and A* Search under different campus map conditions and identify the most suitable algorithm for an autonomous campus shuttle.

2. Explanation of Algorithms

1. Uniform-Cost Search (UCS)

- Expands the node with the lowest path cost.

- Uses the function (f(n)=g(n)).
- Finds the optimal path when all step costs are non-negative.
- Does not use a heuristic.

2. Greedy Best-First Search

- Expands the node that appears closest to the goal.
- Uses (f(n)=h(n)).
- Usually works quickly when the goal is easy to approach.
- Does not always find the shortest path.

3. A Search*

- Combines the actual path cost and estimated remaining cost.
- Uses (f(n)=g(n)+h(n)).
- Finds an optimal path when the heuristic is admissible and the search implementation is correct.
- Balances the cost already travelled and the estimated cost remaining.

3. Designed Heuristic Function

For a grid-based campus map, use Manhattan Distance.

[h(n)=|x_n-x_g|+|y_n-y_g|]

Where:

- (x_n,y_n) = current position.
- (x_g,y_g) = goal position.

Why is it admissible?

Manhattan distance assumes that the shuttle can move horizontally and vertically without obstacles. It ignores construction zones and closed roads.

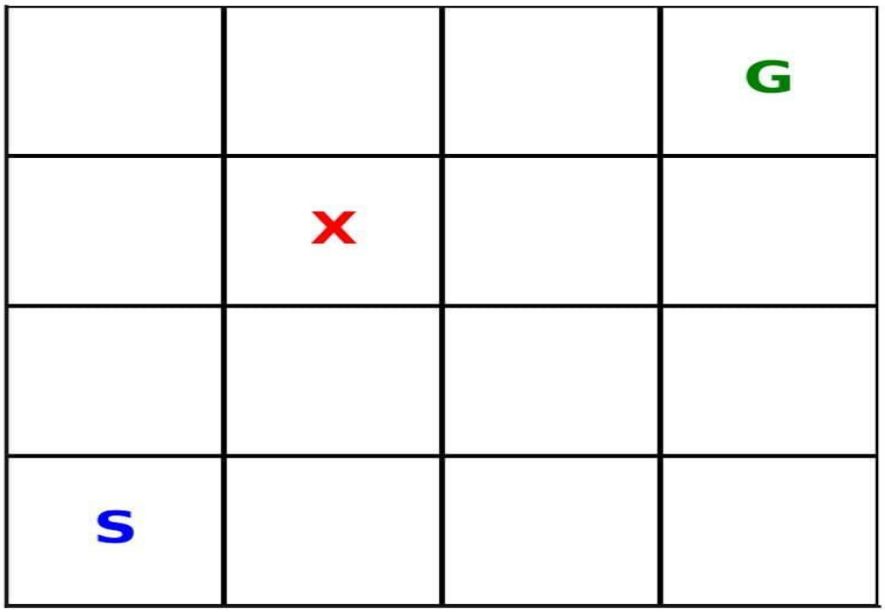
Therefore, the estimated distance is never greater than the actual shortest distance.

[h(n)leq h^*(n)]

Hence, it is an admissible heuristic.

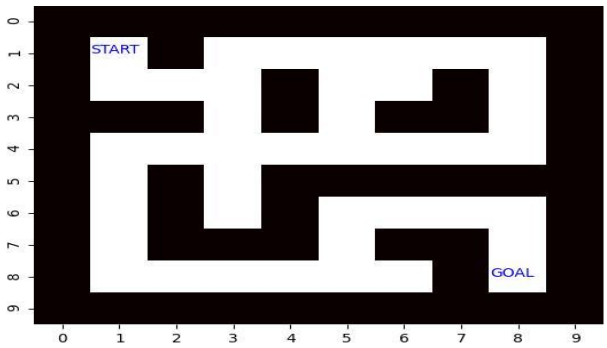
4. Designed Maps

Map 1 – Open Map

- 

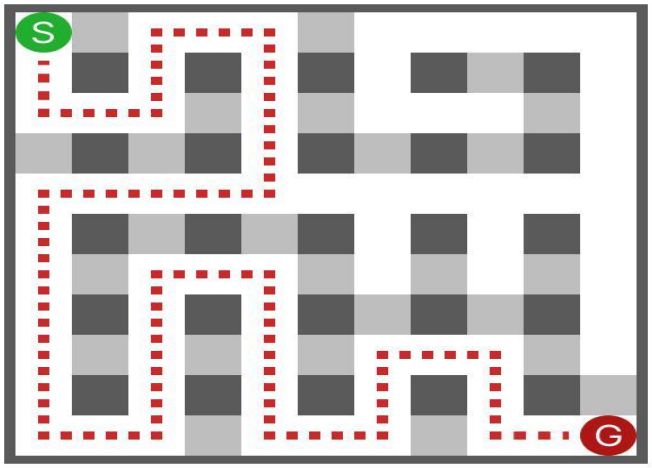
Very few obstacles.
- Direct path is available.
- All algorithms can reach the destination.
- Greedy and A* are expected to explore fewer unnecessary nodes than UCS.

Map 2 – Maze-like Map

- 

- Contains many obstacles.
- The direct path is blocked.
- Greedy may enter misleading corridors.
- UCS explores according to path cost.
- A* uses both path cost and heuristic to find an optimal route.

Map 3 – Dead-end Map



- Contains dead-end paths.
- Greedy may select a dead end because it appears closer to the goal.
- UCS and A* can recover and find the correct path.

5. Predicted Behaviour

Map condition	**UCS**	**Greedy**	**A***
Open map	More exploration	Fast	Fast and optimal
Maze map	Reliable but explores more	May be misleading	Efficient and optimal
Dead-end map	Reliable	Can waste time	Avoids unnecessary exploration

6. Comparison Measures

The following measures should be recorded after implementation:

Measure	**Meaning**
Nodes expanded	Number of nodes explored
Nodes generated	Total nodes created
Maximum frontier size	Maximum pending nodes
Path cost	Total travel cost
Optimality	Whether shortest path is found

Execution time	Time taken by algorithm

7. Design Decision

I recommend A Search* for the campus shuttle.

Reasons:

1. It considers both travelled distance and estimated remaining distance.
2. It finds the optimal path with an admissible heuristic.
3. It performs better than UCS in many maps because it uses additional guidance.
4. It is more reliable than Greedy Search when obstacles and dead ends are present.

Since approximately 95% of shuttle trips occur on open roads, A* provides a good balance between speed and optimality.

Result: A* Search is selected as the preferred algorithm because it provides efficient and optimal route planning.

TASK 2 – When Is an Online Search Agent an Appropriate Engineering Choice? (4 Marks)

1. Aim

To study how an online search agent explores an unknown environment and learns the map while moving towards a charging dock.

2. Explanation

An online search agent makes decisions without knowing the complete environment in advance.

The rover:

- Starts without a complete map.
- Senses only nearby cells.
- Moves into unknown areas.
- Records obstacles and available paths.
- Learns the environment during exploration.

Online Depth-First Search (DFS) is used to explore the facility.

3. Algorithm

1. Start from the initial position.

2. Sense the neighbouring cells.
3. Select an unexplored available cell.
4. Move to that cell.
5. Record the visited cell.
6. If a dead end is reached, return to the previous cell.
7. Continue until the charging dock is found or all reachable cells are explored.

4. Experimental Design

Three experiments are designed.

Experiment	Environment	Expected behaviour
1	Small maze, nearby target	Fewer movements
2	Large maze, distant target	More movements
3	Unreachable target	Explores all reachable cells

5. Performance Measures

Number of moves: Total movements performed by the rover.

Revisited cells: Number of times the rover returns to previously visited cells.

Competitive ratio:

[CR=frac{text{Cost travelled by online agent}}{text{Optimal cost with full map}}]

A lower competitive ratio indicates better performance.

If the target is unreachable, the optimal cost is undefined or infinite, so the competitive ratio cannot be calculated normally. Instead, report total exploration cost and whether the reachable area was completely explored.

6. When Is Online Search Appropriate?

Online search is appropriate when:

- The environment is unknown.
- A complete map is unavailable.
- The environment changes frequently.
- The rover can sense nearby locations.
- Immediate exploration is necessary.

It is not suitable when:

- The environment is extremely large.
- The same environment is explored repeatedly.
- The complete map is already available.
- Exploration consumes too much time and battery power.

7. Changed Condition – 20 Rovers in 20 Facilities

The organisation now uses 20 rovers in 20 facilities, and the maps remain unchanged for months.

My recommendation changes.

Instead of using online DFS for every mission, I recommend:

1. Use online DFS during the first exploration.
2. Store the discovered map.
3. Use the stored map for future missions.
4. Use an efficient route-planning algorithm.
5. Update the stored map only when changes are detected.

LRTA* can also improve decisions through learning across repeated trials.

Result: Online search is useful for the first exploration of unknown environments. For repeated missions in unchanged facilities, learning and storing the map is more efficient.

TASK 3 – Engineer the Behaviour of Alpha-Beta Pruning (6 Marks)

1. Aim

To compare Minimax and Alpha-Beta Pruning and understand how move ordering affects the number of nodes visited in a game tree.

2. Explanation

Minimax Algorithm

Minimax is used in two-player games. It selects the best possible move by assuming that both players play optimally.

- MAX player tries to get the highest score.
- MIN player tries to get the lowest score.

Alpha-Beta Pruning

Alpha-Beta Pruning improves Minimax by avoiding unnecessary branches.

- Alpha (α): Best value found by MAX.

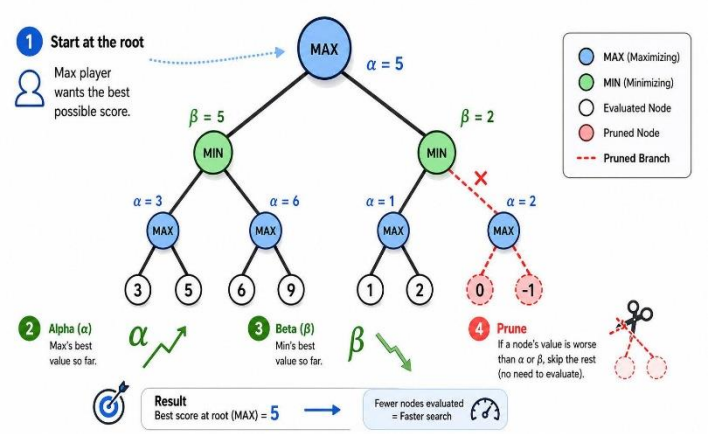
- Beta (β): Best value found by MIN.
- Pruning occurs when:

[alphageqbeta]

3. Designed Test Cases

Test 1 – Good Move Ordering

Consider a game tree where the best moves are examined first.



Expected behaviour:

- Alpha and beta values become restrictive quickly.
- Many unnecessary branches are removed.
- Fewer nodes are visited.

Test 2 – Poor Move Ordering

In this tree, the worst moves are examined first.

Expected behaviour:

- Useful alpha and beta values are discovered late.
- Very few branches are pruned.
- More nodes are visited.

Test 3 – Equal Leaf Values

All leaf nodes contain equal values.

Expected behaviour:

- Alpha and beta values may become equal.
- Depending on the pruning condition and traversal order, cutoffs can occur.
- This test helps study the effect of equal values.

4. Comparison

Measure	Good ordering	Poor ordering
Nodes visited	Low	High
Cutoffs	More	Fewer
Execution time	Less	More
Final decision	Optimal	Optimal

5. Algorithm

1. Start from the root node.
2. Set alpha to negative infinity.
3. Set beta to positive infinity.
4. MAX selects the highest child value.
5. MIN selects the lowest child value.
6. If alpha becomes greater than or equal to beta, stop exploring that branch.
7. Return the final best value.

6. Analysis

A game tree with good move ordering requires less computational effort because useful moves are examined first.

A tree with poor move ordering requires more work because the algorithm cannot prune branches early.

The branching factor and depth alone do not completely determine actual execution effort. Move ordering also has a major influence.

Result: Alpha-Beta Pruning reduces the number of nodes visited by Minimax. Its performance depends strongly on the order in which moves are examined.

TASK 4 – Minimax and the Cost of Examining Every Possibility (6 Marks)

1. Aim

To develop a depth-limited Minimax algorithm and study how the number of examined game positions increases with search depth.

2. Explanation

Minimax examines possible moves of both players and selects the best move.

A depth limit is used to control the number of positions examined.

When the search reaches the depth limit, an evaluation function estimates the quality of the current position.

3. Mathematical Derivation

Let:

- (b) = Average number of legal moves.
- (d) = Search depth.

Total nodes examined:

[N=1+b+b^2+cdots+b^d]

Using the geometric series:

[boxed{N=frac{b^{d+1}-1}{b-1}}]

Number of leaf nodes:

[boxed{L=b^d}]

For example, assume each player has 3 possible moves.

Depth	Total nodes	Leaf evaluations
1	4	3
2	13	9
3	40	27
4	121	81

The number of positions increases exponentially with depth.

4. Evaluation Function

For a simple board game, the evaluation function can be:

[E(s)=3W+2C-2O]

Where:

- W = Number of winning opportunities.
- C = Number of controlled positions.
- O = Number of opponent threats.

A higher score indicates a better position for MAX.

At the depth limit, the evaluation function assigns a score to the current board instead of exploring further.

5. Algorithm

1. Start from the current game position.
2. Generate all possible legal moves.
3. Apply one move.
4. Continue until the depth limit is reached.
5. Calculate the evaluation score.
6. MAX selects the highest score.
7. MIN selects the lowest score.
8. Return the best move.

6. Experimental Design

Test the algorithm using three different search depths with branching factor 3.

Test	Branching factor	Depth	Expected nodes
1	3	2	13
2	3	3	40
3	3	4	121

These are theoretical node counts for a complete tree.

7. Scalability Inquiry

For my experiment, I define an acceptable response time as 1 second.

Plain Minimax becomes impractical when the number of generated positions becomes too large to examine within this limit.

The main problem is exponential growth in the number of nodes.

To improve the system, a redesigned strategy should:

- Avoid unnecessary node exploration.
- Use better move ordering.
- Reuse previously calculated information.
- Reduce evaluation-function calls.

Alpha-Beta Pruning is one possible improvement.

Result: Minimax provides a correct decision when the search tree is completely evaluated to the selected depth. However, its computational cost increases exponentially as depth increases.

TASK 5 – Derive a Constraint-Propagation Algorithm from a Principle (7 Marks)

1. Aim

To develop a constraint-propagation algorithm for university examination timetabling and reduce unnecessary backtracking.

2. Problem Explanation

In a university, examinations that share students cannot be conducted at the same time.

Each exam is considered a variable, and each variable has a domain of available time slots.

Example:

Exam	Available slots
A	12
B	12
C	23

If two exams share students, they must be assigned different slots.

3. Constraint Representation

Let:

- Variables = Examinations.
- Domains = Available time slots.
- Constraints = Exams sharing students cannot use the same slot.

For example:

[Aneq B]

[Bneq C]

These are binary constraints.

4. Constraint Propagation

Constraint propagation removes values that cannot produce a valid solution.

For example:

Initially:

[A={1},quad B={1,2}]

Since A and B cannot have the same slot, B cannot use slot 1.

After propagation:

[B={2}]

This reduces the search space.

5. Pseudocode

START

Create variables and domains

Add constraints between exams

Repeat:

    Check every connected pair

    Remove values without support

    If any domain becomes empty:

        Return FAILURE

Until no domain changes

If all variables are assigned:

    Return SOLUTION

Otherwise:

    Select an unassigned variable

    Try a possible value

Apply propagation

If successful:

Continue search

Otherwise:

Backtrack

STOP

This procedure follows the principle of arc consistency (AC-3).

6. Designed Test Cases

Test 1 – Propagation Solves Without Search

Initial domains:

Exam	Domain
A	1
B	12
C	23

Constraints:

[Aneq B,quad Bneq C]

Propagation:

1. A is fixed to slot 1.
2. Remove slot 1 from B.
3. B becomes slot 2.
4. Remove slot 2 from C.
5. C becomes slot 3.

Final result:

Exam	Final slot

A	1
B	2
C	3

The problem is solved without backtracking.

Test 2 – Failure Detection

Initial domains:

[A={1}, B={1}, C={1}]

Constraints:

[Aneq B,quad Bneq C,quad Aneq C]

All three exams require different slots, but only one slot is available.

Propagation detects an empty domain and reports failure.

7. Comparison

Measure	**Without propagation**	**With propagation**
Nodes visited	More	Fewer
Backtracking	More	Less
Constraint checks	Repeated	Reduced search
Domain wipe-out	Detected during search	Can be detected earlier

Actual counts depend on the chosen test instances.

8. Correctness Explanation

Removing an unsupported value does not remove a valid solution.

A value is removed only when a neighbouring variable has no compatible value.

Therefore, that value cannot be part of any valid assignment satisfying the constraint.

Hence, removing unsupported values preserves all valid solutions.

Result: Constraint propagation reduces the search space and helps solve examination timetabling problems more efficiently.

TASK 6 – Exhaustive Search, Local Search and Combinatorial Growth (8 Marks)

1. Aim

To compare exhaustive search and local search for finding the shortest delivery tour.

2. Problem Explanation

A courier starts from a depot, visits every customer exactly once and returns to the depot.

The objective is to minimise the total travel distance.

This problem is called the Travelling Salesperson Problem (TSP).

3. Exhaustive Search Algorithm

Exhaustive search examines every possible delivery route.

Steps:

1. Start from the depot.
2. Generate all possible customer orders.
3. Calculate the total distance of each route.
4. Compare the route distances.
5. Store the shortest route.
6. Return the optimal route.

For a symmetric distance matrix with n customers excluding the depot, the number of unique tours is:

[boxed{frac{n!}{2}}]

This is because a tour and its reverse have the same distance.

4. Designed Example

Consider a depot and three customers.

Distance matrix:

Location	**A**	**B**	**C**	**Depot**
A	0	10	15	20
B	10	0	35	25
C	15	35	0	30
Depot	20	25	30	0

Example route:

Depot → A → B → C → Depot

Total distance:

[20+10+35+30=95]

Other route:

Depot → A → C → B → Depot

[20+15+35+25=95]

The optimal distance is 95 units.

5. Growth Investigation

The number of unique tours increases factorially.

Customers (n)	Number of tours
3	3
4	12
5	60
6	360
7	2,520
8	20,160
9	181,440
10	1,814,400

This demonstrates the rapid growth of exhaustive search.

6. Local Search Alternative – Hill Climbing with 2-opt

Local search improves an existing route instead of generating every possible route.

The 2-opt method:

1. Generate an initial delivery route.
2. Select two edges.
3. Exchange their connections to create a new route.
4. Calculate the new distance.
5. If the distance decreases, accept the new route.
6. Repeat until no improvement is found.

Random restarts can be used to reduce the possibility of getting stuck in a local optimum.

7. Comparison

Feature	Exhaustive Search	Local Search
Search method	All possible routes	Nearby routes
Optimal solution	Guaranteed	Not guaranteed
Execution time	Very high for large n	Usually lower
Memory	Can be high	Usually lower
Large problems	Not suitable	More suitable

8. Percentage Error

The quality of the local-search result can be calculated using:

[text{Error %}= frac{text{Local distance}-text{Optimal distance}} {text{Optimal distance}}times100]

For example, if the optimal route is 95 and the local-search route is 100:

[text{Error}=frac{100-95}{95}times100]

[=5.26%]

9. Deployment Decision

I select a maximum response time of 1 second for the delivery planner.

My deployment strategy is:

- Use exhaustive search for small customer sets where it completes within the response-time limit.
- Use local search for larger customer sets.
- Compare local-search results with known optimal solutions on smaller instances.
- Use multiple random restarts to improve solution quality.

For this experimental design, I would initially target exhaustive search up to 8 customers and validate the one-second limit on the actual system before deployment.

Result: Exhaustive search guarantees the optimal delivery route but becomes expensive as the number of customers increases. Local search provides a faster alternative for larger problems.

TASK 7 – Diagnose, Redesign and Validate (13 Marks)

Selected Problem: Delivery Tour Planning (Task 6)

I selected the Travelling Salesperson Problem because exhaustive search becomes computationally expensive when the number of customers increases.

A. Diagnose the Computational Bottleneck (2 Marks)

The main limitation of exhaustive search is that it generates and evaluates every possible delivery route.

The number of unique routes is:

[N=frac{n!}{2}]

Problems:

1. The number of possible routes increases factorially.
2. Many routes are unnecessarily examined.
3. Computational time increases as customers are added.
4. Memory and processing requirements also increase.

Main bottleneck: Repeated evaluation of routes that cannot produce a better result than the best route already found.

B. Define the Design Objective (2 Marks)

The redesigned system should:

1. Reduce the number of routes explored.
2. Avoid unnecessary distance calculations.
3. Maintain the ability to find the optimal route.
4. Produce results within the selected response time.

Measurable objective: Reduce the number of complete routes evaluated compared with exhaustive search while maintaining the same optimal path cost.

C. Design the Revised Computational Strategy (4 Marks)

Proposed Strategy – Branch and Bound with a Lower-Bound Estimate

Instead of exploring every complete route, the redesigned system estimates the minimum possible cost of completing a partial route.

If the estimated minimum cost is already greater than or equal to the best complete route, that branch is stopped.

The lower bound can be calculated using:

[LB=C+MST(U)+d(a,U)+d(U,D)]

Where:

- C = Current route cost.
- MST(U) = Minimum spanning tree cost of unvisited customers.
- d(a,U) = Minimum connection from the current location to an unvisited customer.
- d(U,D) = Minimum connection from an unvisited customer to the depot.
- U = Set of unvisited customers.
- D = Depot.

If:

[LBgeq BestCost]

The branch is pruned.

Pseudocode

START

Find an initial route using nearest neighbour

Store its cost as BestCost

Generate partial delivery routes

For each partial route:

    Calculate current cost

    Calculate lower-bound cost

    If lower bound >= BestCost:

        Stop exploring this branch

Else if all customers are visited:

Return to depot

If route cost < BestCost:

Update BestCost

Else:

Select an unvisited customer

Continue exploring

Display the best route and distance

STOP

Advantages

- Avoids exploring unnecessary routes.
- Uses an existing best route to guide the search.
- Can still find the optimal route if the lower bound is valid.
- Reduces complete route evaluations.

Trade-offs

- Requires additional memory to store partial routes.
- Lower-bound calculations need extra processing.
- Performance depends on the quality of the initial route and lower bound.

D. Predict Before Implementing (1 Mark)

Before implementation, I predict:

Measure	**Exhaustive search**	**Redesigned search**
Routes explored	All possible	Selected branches
Complete route evaluations	High	Lower

Execution time	High	Expected to decrease
Optimality	Guaranteed	Guaranteed with valid bounds and complete search
Memory	Depends on implementation	Additional search information

I expect the redesigned algorithm to reduce the number of nodes explored and complete route evaluations.

E. Implement and Validate the Redesign (2 Marks)

Use the same distance matrix and customer locations for both algorithms.

Record the following:

Measure	**Original algorithm**	**Redesigned algorithm**
Nodes expanded	Record from execution	Record from execution
Complete routes evaluated	Record from execution	Record from execution
Best route distance	Record from execution	Record from execution
Pruned branches	0	Record from execution
Execution time	Record from execution	Record from execution

Validation procedure:

1. Run exhaustive search on a small delivery problem.
2. Record its optimal distance.
3. Run the redesigned algorithm using the same input.
4. Compare both route distances.
5. Compare nodes expanded and complete routes evaluated.
6. Verify that both algorithms produce the same optimal distance.

F. Defend the Final Design (2 Marks)

The redesigned algorithm is expected to perform better because it avoids searching branches that cannot improve the current best solution.

Expected benefits:

- Fewer unnecessary route evaluations.
- Reduced computational effort.

- Better scalability for larger delivery problems.
- Optimal solution retained through valid lower-bound pruning.

Limitations:

- Calculating lower bounds requires additional processing.
- A weak lower bound may not prune many branches.
- For very small problems, exhaustive search may be simpler and faster.

Result: The redesigned Branch and Bound strategy provides a more efficient approach to delivery tour planning by reducing unnecessary search while preserving optimality.

Overall Conclusion

Artificial Intelligence search algorithms help solve complex problems by exploring possible states and selecting suitable solutions.

Through these seven tasks, the following concepts are studied:

Task	Main concept	Important learning
1	UCS, Greedy and A*	Selecting suitable pathfinding algorithms
2	Online Search	Exploring unknown environments
3	Alpha-Beta Pruning	Reducing unnecessary game-tree exploration
4	Minimax	Understanding exponential growth
5	Constraint Propagation	Reducing search through domain elimination
6	Exhaustive and Local Search	Solving optimisation problems
7	Algorithm Redesign	Improving computational efficiency

Final conclusion: The assignment demonstrates how different AI algorithms solve real-world problems and how their efficiency can be improved through mathematical analysis, experimentation and redesign.